

Comparative Fault Localization Analysis Report

Describer-Decider v34.0 — Full Dataset Validation (7 Own Traces + 10 Group 411 Cases)

1. Executive Summary

Metric	Our Traces (7 apps)	Group 411 Traces (10 cases)	Combined (17 apps)
Total apps evaluated	7	10	17
Total steps in dataset	86	119	205
Steps skipped (non-VLM / wait)	5	0	5
Steps analyzed	81	119	200
Ground truth failures	12	20	32
System-predicted failures	13	22	35
True Positives (TP)	12	18	30
False Positives (FP)	1	4	5
False Negatives (FN)	0	2	2
Precision	92.3%	81.8%	85.7%
Recall	100.0%	90.0%	93.8%
F1 Score	96.0%	85.7%	89.6%

Note on step counts: Our "analyzed" count counts only VLM-processed steps (skipped steps removed). For the 86 total steps in our 7 traces, 5 are skipped (Bitaksi: 1, Flight App: 3, Yemeksepeti: 1), leaving **81 analyzed**. Group 411 traces have no skipped steps: all 119 are analyzed across 10 cases.

Bottom line (Our Traces): v34.0 achieves **100% recall** on all 7 proprietary test cases — every real failure was detected. Precision dropped to **92.3%** from the perfect 100% in v33.0 due to one new false positive: Flight App Step 19 (click on BASIC package option incorrectly classified as `action_failed`). This FP is a non-deterministic GPT-4o output — the step correctly passed in v33.0. All other systematic issues remain resolved.

Bottom line (Group 411 Traces — Extended to 10 cases): Adding cases 6–10 (73 new steps, 7 real failures) changes the aggregate Group 411 metrics to **81.8% precision** and **90.0% recall** (F1 = 85.7%). The 5 new cases contributed 7 TP and 3 FP with 0 new FN: Cases 6, 7, and 10 are perfect (100%/100%/100%); Case 8 (Car App) introduced 2 FPs from abstract presence-check instructions misclassified as `element_missing`; Case 9 (Hotel Booking App) added 1 FP from the same pattern. The original 2 FNs (Bank App Step 6, Rent-a-Car Step 8) remain unchanged architectural gaps.

Combined overall F1: 89.6% — the original 5 Group 411 cases metrics (91.7% precision / 84.6% recall / 88.0% F1) are preserved within the extended dataset. The new dominant FP pattern across cases 8 and 9 is **abstract state/presence verification instructions** (e.g. "Uygulamanın açıldığını doğrula", "Otellerin yüklendiğini doğrula") being misclassified as **element_missing** when the instruction describes a system state rather than a named UI element.

2. Ground Truth — Confirmed Real Failures

2A. Our Own Traces

App	Step	Failure Description	Evidence Type
Migros	Step 7	POST /api/v1/cart/confirm → 400 Bad Request; cart state inconsistency (local: 1 item, server: EMPTY); UI showed empty cart	Log: status: FAILED + network ERROR
Bitaksi	—	No failures	All StepRunner markers: PASSED
Clock	—	No failures	All steps progressed normally; no log errors
Flight App	Step 18	Click on 12.15→13.35 flight item did not select it; wrong flight context (21:15) loaded	Visual: screen shows 21:15 departure in POST
Flight App	Steps 20–24	Cascading content_mismatch: each verification step confirms 21:15 instead of 12:15	Visual: expected flight time never appears
QNB	Step 4	Click on Günlük option produced no UI state change; button remained unselected	Visual: no change between PREV/POST
Yemeksepeti	Step 7	Click on Mixed (Medium) Pizza opened Mediterranean (Central) detail page	Visual + Log: wrong item in POST
Yemeksepeti	Step 14	AbstractVerification confirmed wrong pizza persists to checkout	Visual: Mediterranean (Central) in order summary
O Bilet	Step 7	POST /api/v3/bus/reserve-seat → 504 Gateway Timeout; reservation service unresponsive	Log: NET_FAIL + SYS_ERR
O Bilet	Step 8	Navigation to PassengerDetailsActivity blocked by failed reservation dependency	Log: SYS_ERR + UI_STALL

Total: 12 real failures across 7 apps.

2B. Group 411 Traces

App	Step	Failure Description	Evidence Type
Passo App	Step 10	Click on occupied (black) seat — no state change; seat remained unselectable	Visual: no change between PREV/POST
Bank App	Step 4	POST /transfer → 500 Internal Server Error ; transfer silently failed on backend	Log: NET_FAIL + SYS_ERR
Bank App	Step 6	Dashboard shows balance of 5250.00 TL unchanged after a "successful" transfer — systematic inconsistency between displayed success and actual server rejection	Cross-step state: balance should update after Step 5 success screen
Bank App	Step 7	Abstract verification of balance — expected value not confirmed in POST; balance of 5250.00 TL is unchanged despite Step 4's failed transfer (should reflect deduction)	Visual: balance unchanged; content mismatch
Ticket App	Step 2	Click on Theater category — no UI response or navigation occurred	Visual: no change between PREV/POST
Ticket App	Step 3	RenderFlex overflow by 869 pixels on right side of Sports Events screen	Log: rendering exception in console
Ticket App	Step 6	Click on occupied seat A3 — system error Seat unavailable ; seat not selectable	Log: SYS_ERR; Visual: no change
Ticket App	Step 10	POST /api/v1/payment → 500 Internal Server Error ; payment could not be processed	Log: NET_FAIL + SYS_ERR
Rent-a-Car	Step 5	Click on Kiral for Mercedes-Benz G-Class blocked — corporate account required for intercity Izmir drop-off	Log: SYS_ERR
Rent-a-Car	Step 8	Promo code DISCOUNT20 entered and UI claimed discount applied, but total price remained ₺17,000 — visible price-discount inconsistency missed by the observer	Visual: discount message shown but total unchanged
Rent-a-Car	Step 9	Discount message " Tebrikler! %20 indirim uygulandı! " appeared but total remained ₺17,000 — content mismatch on amount	Visual: amount unchanged
Rent-a-Car	Step 10	POST /payment → 504 Gateway Timeout on Izmir regional gateway	Log: NET_FAIL + SYS_ERR
Wikipedia	Step 4	Instruction target Osmanlı padişahı 3. Murad tahta çıktı seç. not present in any visible UI element	Visual: element absent; instruction unrelated to current page context
Video App	Step 15	Episode description displayed in Turkish instead of English after language-change-to-English action;	Visual: content language mismatch

App	Step	Failure Description	Evidence Type
		language setting did not propagate to episode detail page	
Investment App	Step 7	POST /trade/order → 504 Gateway Timeout; trade order failed silently on backend while UI showed a success overlay	Log: NET_FAIL + SYS_ERR
Investment App	Step 14	AbstractVerification: portfolio shows empty holdings (0 THYAO shares) after failed trade; expected 100 shares to appear	Visual: empty portfolio; Log: 200 OK with empty holdings array
Car App	Step 13	AbstractVerification: Sipariş Özeti (Order Summary) screen does not confirm 'M Sport Paket' equipment — standard equipment shown instead	Visual: M Sport absent from summary
Hotel Booking App	Step 5	Guest selection UI confirmed '2 Yetişkin, 2 Çocuk' but backend POST payload saved 1 adult, 0 children — UI-backend data mismatch	Log: POST 200 OK with incorrect payload
Hotel Booking App	Step 15	AbstractVerification: reservation summary screen shows '1 Yetişkin' instead of expected '2 Yetişkin, 2 Çocuk'	Visual: guest count mismatch
Clothing App	Step 11	AbstractVerification: product detail page shows color 'Kırmızı' and size 'XL' instead of expected 'Siyah'/'M' from applied filters	Visual: filter attributes not reflected in product detail

Total: 20 real failures across 10 cases.

3. Part A — Our Own Traces: Case-by-Case Breakdown

App 1 — Migros · 8 steps · Log: nested (preassigned)

Scenario: App splash → main dashboard → Categories tab → search "Raffaello" → add to cart → select delivery address → view cart → confirm order.

Step	Action / Element	System Result	Ground Truth	Assessment
0	waitUntil — Migros Splash Logo	PASS	Pass ✓	TN
1	waitUntil — Service Selection Dashboard	PASS	Pass ✓	TN
2	click — Categories tab	PASS	Pass ✓	TN
3	type — Search "Raffaello"	PASS	Pass ✓	TN
4	click — Add to Cart (Ferrero Raffaello 150G)	PASS	Pass ✓	TN

Step	Action / Element	System Result	Ground Truth	Assessment
5	click — Yurt delivery address	PASS	Pass ✓	TN
6	click — Cart icon	PASS	Pass ✓	TN
7	click — Confirm button	FAIL (system_error)	Fail X	TP ✓

Result: 1 failure detected, 1 real failure → Perfect precision and recall.

True Positive — Step 7: The cart confirmation failure was definitively detected. Log evidence: `POST /api/v1/cart/confirm → 400 Bad Request (Cart is empty), STATE_MISMATCH: Local cart state shows 1 item but session validation returned EMPTY`. The visual observer confirmed the cart appeared empty in POST with the Confirm button absent. The `[NET_FAIL, SYS_ERR]` flags correctly elevated the verdict to `system_error`. Classified as **ISOLATED FAILURE** — no upstream cause was found, correctly reflecting that the root is a backend session state inconsistency, not a cascade from prior steps.

Performance vs. v33.0: Identical. No change in this app.

App 2 — Bitaksi · 9 steps (1 skipped) · Log: flat_assertions

Scenario: App launch (waitUntil, skip) → tap "Where to?" → save home address → select Lux ride → profile info → save profile → trip history → payment methods → delete card.

Step	Action / Element	System Result	Ground Truth	Assessment
0	waitUntil — Main map screen [VLM=false]	SKIPPED	Pass (skip)	Correct skip
1	click — "Where to?" search input	PASS	Pass ✓	TN
2	click — Save (home address)	PASS	Pass ✓	TN
3	click — Lux ride option	PASS	Pass ✓	TN
4	click — Profile information	PASS	Pass ✓	TN
5	click — Save button	PASS	Pass ✓	TN
6	click — Trip history	PASS	Pass ✓	TN
7	click — Payment methods	PASS	Pass ✓	TN
8	click — Delete card	PASS	Pass ✓	TN

Result: 0 failures detected, 0 real failures → Perfect TN.

Performance vs. v33.0: Identical. No change in this app.

App 3 — Clock · 7 steps · Log: flat_indexed

Scenario: Enable alarm toggle → navigate to Stopwatch tab → start stopwatch → reset stopwatch → scroll minute picker → pause timer → navigate to World Clock tab.

Step	Action / Element	System Result	Ground Truth	Assessment
0	click — Alarm toggle (08:17)	PASS	Pass ✓	TN
1	click — Stopwatch tab	PASS	Pass ✓	TN
2	click — Start button	PASS	Pass ✓	TN
3	click — Reset button	PASS	Pass ✓	TN
4	scroll — Minute picker	PASS	Pass ✓	TN
5	click — Pause button	PASS	Pass ✓	TN
6	click — World Clock tab	PASS	Pass ✓	TN

Result: 0 failures detected, 0 real failures → Perfect TN.

Performance vs. v33.0: Identical. No change in this app.

App 4 — Flight App · 25 steps (3 skipped) · Log: dynamic (keyword-based)

Scenario: App splash → waitUntil (×2, skip) → dismiss X icon → verify login screen → guest session → allow notification permission → select departure city Nereden → pick İzmir → select destination → pick Ankara → open departure calendar → select date 21 → search flights → wait (skip) → select flight class → pick flight time → search → click flight 12.15→13.35 → select BASIC package → abstractVerification (×5 steps).

Step	Action / Element	System Result	Ground Truth	Assessment
0	waitUntil — "Tüm seyahatin tek uygulamada"	PASS	Pass ✓	TN
1–2	waitUntil steps [VLM=false]	SKIPPED	Pass (skip)	Correct skip
3	click — Çarpı (X icon)	PASS	Pass ✓	TN
4	verify — Login screen	PASS	Pass ✓	TN
5	click — Üye olmadan devam et	PASS	Pass ✓	TN
6	click — İzin Ver (notifications)	PASS	Pass ✓	TN
7	click — Nereden (departure city)	PASS	Pass ✓	TN
8	click — İzmir	PASS	Pass ✓	TN
9	click — Arrival city input	PASS	Pass ✓	TN
10	click — Ankara	PASS	Pass ✓	TN
11	click — Departure date field	PASS	Pass ✓	TN

Step	Action / Element	System Result	Ground Truth	Assessment
12	click — Calendar date 21	PASS	Pass ✓	TN
13	click — Search button (UCUZ BILET BUL)	PASS	Pass ✓	TN
14	wait [action=wait]	SKIPPED	Pass (skip)	Correct skip
15	click — Sort by departure time	PASS	Pass ✓	TN
16	click — Evening flights filter	PASS	Pass ✓	TN
17	click — Show more flights	PASS	Pass ✓	TN
18	click — 12.15→13.35 flight item	FAIL (content_mismatch)	Fail ✗	TP ✓
19	click — BASIC package option	FAIL (action_failed)	Pass ✓	FP ✗
20	abstractVerify — flight time 12:15–13:35	FAIL (content_mismatch)	Fail ✗	TP ✓
21	abstractVerify — BASIC + 12:15	FAIL (content_mismatch)	Fail ✗	TP ✓
22	abstractVerify — flight time 12:15–13:35	FAIL (content_mismatch, caused by Step 21)	Fail ✗	TP ✓
23	abstractVerify — 12:15 departure + BASIC	FAIL (content_mismatch, caused by Step 22)	Fail ✗	TP ✓
24	abstractVerify — 12:15–13:35 final confirm	FAIL (content_mismatch, caused by Step 23)	Fail ✗	TP ✓

Result: 6 TP, 1 FP, 0 FN — 6 real failures detected, 1 passing step incorrectly flagged.

True Positive — Step 18 (Root Failure): The click on the 12.15→13.35 flight item loaded a different flight context. The visual observer noted the flight class selection overlay appeared with no sign of the targeted time slot. Classified as **content_mismatch** — the element was present in PREV (list view) but the POST screen showed the wrong flight context (21:15 departure). Identical to v33.0 detection.

True Positives — Steps 20–24 (Cascading Chain): All five downstream verification steps correctly flagged the same root cause: the screen consistently shows a 21:15–22:35 departure/arrival pair instead of the expected 12:15–13:35. In v34.0, the causal chain runs **Step 18 → Step 19 → Step 20 → Step 21 → Step 22 → Step 23 → Step 24** (7 steps). Since Step 19 is a false positive, the formal chain is mis-routed through an incorrect link — the real root remains Step 18.

False Positive — Step 19 (New in v34.0): The click on the BASIC package option within the flight class overlay was classified as **action_failed**. The system observed: "The screen remains on the flight class selection with 'BASIC' option visible, showing baggage details and no additional cost" — interpreting the unchanged overlay view as a non-response. However, this is a misread: the BASIC package was successfully selected within the (wrong-flight) overlay context. Rule 3's CONTENT EXPANSION SUCCESS clause should have

fired — the flight class overlay in POST is new content produced by the Step 18 click, and Step 19's selection within it constitutes a valid action with a registered response. The Decider instead applied Rule 1 (no state change visible), producing an incorrect **action_failed** verdict. This step correctly passed in v33.0. The regression is consistent with GPT-4o temperature=0 non-determinism in boundary cases where overlay content and state-change evidence are ambiguous.

Notes on dynamic log slicing: Same behavior as v33.0 — keyword-based slicing assigns 5 log entries per step consistently. Log evidence is supportive but not decisive; failures are driven by visual evidence.

Performance vs. v33.0: 1 new FP at Step 19 (was TN); chain extended from 6 steps to 7; net result: 6 TP, 1 FP (was 6 TP, 0 FP).

App 5 — QNB · 13 steps · Log: flat_indexed

Scenario: Open Kartlar → navigate Kartlar → tap Banka Kartlarım → tap Back → open Limit Geçerlilik Süresi → select Günlük → close dropdown → navigate back → open Banka Kartı Ayarları → tap Güncelle → enter limit value → tap Uygula → confirm (Onayla).

Step	Action / Element	System Result	Ground Truth	Assessment
0	click — Kartlar menu	PASS	Pass ✓	TN
1	click — Banka Kartlarım	PASS	Pass ✓	TN
2	click — Back (to main Kartlar)	PASS	Pass ✓	TN
3	click — Limit Geçerlilik Süresi	PASS	Pass ✓	TN
4	click — Günlük option	FAIL (action_failed)	Fail ✗	TP ✓
5	click — Günlük (select)	PASS	Pass ✓	TN
6	dropdown closes; Günlük confirmed	PASS	Pass ✓	TN
7	click — Back (to Kartlar menu)	PASS	Pass ✓	TN
8	click — Banka Kartı Ayarları	PASS	Pass ✓	TN
9	click — Güncelle button	PASS	Pass ✓	TN
10	tap — İşlem Limitiniz input field	PASS	Pass ✓	TN
11	tap — Uygula button	PASS	Pass ✓	TN
12	tap — Onayla (confirmation popup)	PASS	Pass ✓	TN

Result: 1 failure detected, 1 real failure → Perfect precision and recall.

True Positive — Step 4: The click on the **Günlük** option produced no observable UI state change — the option remained visible but neither selected nor highlighted. Classified as **action_failed**. The log evidence (entry: "tapped Limit Geçerlilik Süresi modal opened") confirmed a modal was triggered but the Günlük item itself was not responsive. Correctly isolated (not cascaded) since Step 5 successfully selected Günlük using a different interaction.

Performance vs. v33.0: Identical. No change in this app.

App 6 — Yemeksepeti · 15 steps (1 skipped) · Log: nested (no logs field)

Scenario: Splash screen → waitUntil sign-up (skip) → close Flash Deals banner → open address bar → select delivery address → search "little ceasers pizza" → open Little Caesars → select Mixed (Medium) Pizza → select Normal Dough → select Less Pizza Sauce → remove ingredients → add to cart → view cart → confirm basket → abstractVerification of checkout.

Step	Action / Element	System Result	Ground Truth	Assessment
0	waitUntil — Yemeksepeti Logo	PASS	Pass ✓	TN
1	waitUntil — Sign-up screen [VLM=false]	SKIPPED	Pass (skip)	Correct skip
2	click — Close Banner	PASS	Pass ✓	TN
3	click — Address Bar	PASS	Pass ✓	TN
4	click — Orta Sabanci Univ. A4 address	PASS	Pass ✓	TN
5	type — "little ceasers pizza"	PASS	Pass ✓	TN
6	click — Little Caesars Pizza card	PASS	Pass ✓	TN
7	click — Mixed (Medium) Pizza	FAIL (content_mismatch)	Fail X	TP ✓
8	click — Normal Dough	PASS	Pass ✓	TN
9	click — Less Pizza Sauce	PASS	Pass ✓	TN
10	click — Remove mushrooms & black olives	PASS	Pass ✓	TN
11	click — Add to cart	PASS	Pass ✓	TN
12	click — View your cart	PASS	Pass ✓	TN
13	click — Confirm basket	PASS	Pass ✓	TN
14	abstractVerify — Mixed (Medium) in checkout	FAIL (content_mismatch)	Fail X	TP ✓

Result: 2 failures detected, 2 real failures → Perfect precision and recall.

True Positive — Step 7: The click on "Mixed (Medium) Pizza" opened the "Mediterranean (Central)" pizza detail page. The visual observer correctly identified the wrong item in POST. The system operated in **vision-only mode** for the entire app (all 15 log buckets were empty per the nested format warning), demonstrating that the VLM pipeline alone is sufficient when the visual signal is unambiguous.

True Positive — Step 14: The abstract verification confirmed the wrong pizza persisted to checkout. Steps 7 and 14 are reported as two **isolated failures** — `find_likely_cause()` scores below 0.3 due to the 6-step temporal gap and action type difference (click vs. abstractVerification). This is an unchanged sub-optimality in

chain reporting from v33.0 but does not affect detection accuracy. The root cause analysis section correctly identifies the Step 7 → Step 14 causal relationship even though the formal chain builder does not link them.

Performance vs. v33.0: Identical. No change in this app.

App 7 — O Bilet · 9 steps · Log: nested (preassigned)

Scenario: Splash screen → category dashboard → set Bus + route Istanbul→Ankara (10 Apr) → search → open filter → apply company filter → select 22:00 AKSU trip → select Seat 12 + confirm → waitUntil Passenger Details form.

Step	Action / Element	System Result	Ground Truth	Assessment
0	waitUntil — App Logo	PASS	Pass ✓	TN
1	waitUntil — Category Dashboard	PASS	Pass ✓	TN
2	click — Bus tab + set parameters	PASS	Pass ✓	TN
3	click — SEARCH button	PASS	Pass ✓	TN
4	click — FILTER button	PASS	Pass ✓	TN
5	click — Apply multi-company filter	PASS	Pass ✓	TN
6	click — 22:00 AKSU trip	PASS	Pass ✓	TN
7	click — Seat 12 + Confirm and Continue	FAIL (system_error)	Fail ✗	TP ✓
8	waitUntil — Passenger Details Input	FAIL (element_missing)	Fail ✗	TP ✓

Result: 2 failures detected, 2 real failures → Perfect precision and recall.

True Positive — Step 7 (Root Failure): The system correctly identified the 504 Gateway Timeout on `POST /api/v3/bus/reserve-seat`. Log observer flagged `[NET_FAIL, SYS_ERR]` and the decider classified `system_error`. Visual: loading overlay persisted in POST (reservation did not complete).

True Positive — Step 8 (Cascading Failure): The waitUntil for Passenger Details form correctly failed. Log: `UI_STALL: Navigation to PassengerDetailsActivity blocked by failed reservation dependency`. The causal chain `Step 7 → Step 8` was correctly constructed. This is the cleanest two-step chain in the dataset.

Performance vs. v33.0: Identical. No change in this app.

4. Part B — Group 411 Traces: Case-by-Case Breakdown

Case 1 — Passo App · 11 steps · No unified log (step-level analysis)

Scenario: Verify Futbol category → click Futbol → tap Search → type "Eskişehirspor" → click match → click Bilet Al → increase ticket count → click Bilet Al (block selection) → select block 102 → click Koltuk Seç → click occupied (black) seat.

Ground truth per 411 reference output: Only Step 10 (their step_index 11: occupied seat click → no state change) is a real failure.

Step	Action / Element	System Result	Ground Truth	Assessment
0	abstractVerify — Futbol category visible	PASS	Pass ✓	TN
1	click — Futbol category	PASS	Pass ✓	TN
2	click — Search button	PASS	Pass ✓	TN
3	type — "Eskişehirspor"	PASS	Pass ✓	TN
4	click — Eskişehirspor - Tire 2021 FK	PASS	Pass ✓	TN
5	click — Bilet Al button	PASS	Pass ✓	TN ✓
6	click — (+) ticket quantity	PASS	Pass ✓	TN
7	click — Bilet Al → block selection	PASS	Pass ✓	TN
8	click — Block 102	PASS	Pass ✓	TN
9	click — Koltuk Seç button	PASS	Pass ✓	TN
10	click — Occupied (black) seat	FAIL (action_failed)	Fail X	TP ✓

Result: 1 failure detected, 1 real failure → Perfect precision and recall.

True Positive — Step 10: The click on the black (occupied) seat produced no state change between PREV and POST. The system correctly classified this as `action_failed`. The 411 reference confirms: "No visual change detected after clicking the seat. (Root Cause: The click action may not have been registered, or the seat is not selectable.)" Both systems agree on this failure.

Improvement vs. v33.0 — Step 5 (was FP, now TN): In v33.0, Step 5 was a false positive — the system misread the POST screenshot after clicking `Bilet Al`, interpreting the ticket category list expansion as the button becoming grayed out (inactive). In v34.0, Step 5 correctly passes: "The button state changed to inactive, indicating the click was registered and a prerequisite action is needed." The system now correctly identifies the `Bilet Al` button state change as a valid UI response (Rule 0C TARGET STATE CHANGE SUCCESS), eliminating the VLM content-expansion observation error that produced the FP in v33.0. This is the primary improvement in Group 411 performance for v34.0.

Case 2 — Bank App · 8 steps · Log: inline (g411 format)

Scenario: Verify balance (5250 TL) → navigate to Para Gönder → enter recipient "Tuna" → enter amount "300" → click Transfer → see Transfer Successful screen → navigate to dashboard → abstractVerify balance.

Ground truth per user annotation: Steps 4, 6, and 7 are real failures. Step 6 is a "systematic problem not about rules."

Step	Action / Element	System Result	Ground Truth	Assessment
0	abstractVerify — balance 5250.00 TL	PASS	Pass ✓	TN
1	click — Para Gönder	PASS	Pass ✓	TN
2	type — "Tuna" in Alıcı adı	PASS	Pass ✓	TN
3	type — "300" in amount field	PASS	Pass ✓	TN
4	click — Transfer button	FAIL (system_error)	Fail ✗	TP ✓
5	see — Transfer Successful screen	PASS	Pass ✓	TN
6	navigate — Back to dashboard	PASS	Fail ✗	FN ✗
7	abstractVerify — balance unchanged	FAIL (content_mismatch)	Fail ✗	TP ✓

Result: 2 failures detected, 3 real failures → 2 TP + 0 FP + 1 FN.

True Positive — Step 4: The Transfer button triggered a `500 Internal Server Error`. Log observer flagged `[NET_FAIL, SYS_ERR]`. The system correctly identified this as `system_error` despite the visual POST screen showing a "Transfer Successful" confirmation — the log evidence overrode the misleading visual (log can only override PASS→FAIL, never FAIL→PASS — functioning as intended).

True Positive — Step 7 (failure type changed from v33.0): In v34.0, the abstract verification of balance is classified as `content_mismatch` rather than v33.0's `element_missing`. The system observed: "The screen shows the same balance of 5250.00 TL in both PREV and POST, but the value was not explicitly confirmed in POST." Logs confirmed the balance fetch returned 5250.00 TL — unchanged after the failed transfer. The root cause analysis correctly links Step 4 (500 Server Error) → Step 7 (stale balance), though the formal failure chain builder still reports them as isolated. Both v33.0 and v34.0 correctly identify Step 7 as a TP; the classification shift from `element_missing` to `content_mismatch` is a difference in how the Decider interprets the balance observation, not a correctness change.

False Negative — Step 6 (Systematic Problem — Unchanged from v33.0): The dashboard shows the balance of 5250.00 TL unchanged after the failed transfer. This is only detectable by comparing the current balance against the expected post-transfer value (5250 – 300 = 4950 TL) derived from Steps 0 and 3. The system passed Step 6 because navigating back to the dashboard appeared visually correct. No single-step visual evidence distinguishes a correct balance from an incorrect one without knowing the expected post-transfer value. This is a genuine **cross-step state inconsistency** that requires expected-state propagation across non-adjacent steps — an architectural gap unchanged from v33.0.

Case 3 — Ticket App · 11 steps · Log: inline (g411 format)

Scenario: Verify Login screen → login (enter credentials) → click Theater category → click Sports category → click Details (Madrid Falcons) → click Select Seats → click occupied seat A3 → click available seat A1 → click Go to Cart → verify seat + price → click Confirm & Pay.

Ground truth per 411 reference output: Steps 2, 3, 6, and 10 are real failures.

Step	Action / Element	System Result	Ground Truth	Assessment
0	abstractVerify — Login screen present	PASS	Pass ✓	TN
1	click — Enter credentials + Login	PASS	Pass ✓	TN
2	click — Theater category	FAIL (action_failed)	Fail X	TP ✓
3	click — Sports category	FAIL (system_error, caused by Step 2)	Fail X	TP ✓
4	click — Details (Madrid Falcons)	PASS	Pass ✓	TN
5	click — Select Seats	PASS	Pass ✓	TN
6	click — Occupied seat A3	FAIL (action_failed)	Fail X	TP ✓
7	click — Available seat A1	PASS	Pass ✓	TN
8	click — Go to Cart	PASS	Pass ✓	TN
9	abstractVerify — seat A1, price \$99.50	PASS	Pass ✓	TN
10	click — Confirm & Pay	FAIL (system_error, caused by Step 6)	Fail X	TP ✓

Result: 4 failures detected, 4 real failures → Perfect precision and recall.

True Positive — Step 2: Click on Theater category produced no visual change between PREV and POST. Log confirmed: "User tapped Theater." — the action was registered but the UI did not navigate. Classified as **action_failed**. Matches 411 reference.

True Positive — Step 3 (Cascading): Sports category navigation showed a RenderFlex overflow (869 pixels on the right side). Log: console rendering exception. Classified as **system_error**. The causal chain **Step 2 → Step 3** was correctly constructed.

True Positive — Step 6: Click on occupied seat A3 returned **ERROR: Seat unavailable**. No UI state change. Classified as **action_failed**. Matches 411 reference exactly.

True Positive — Step 10 (Cascading): **POST /api/v1/payment → 500 Internal Server Error**. Classified as **system_error**. The causal chain **Step 6 → Step 10** was constructed — the occupied-seat error likely corrupted the cart/reservation state, preventing payment completion.

Two failure chains correctly identified:

- **Chain 1:** **Step 2 → Step 3** (UI non-response → rendering error)
- **Chain 2:** **Step 6 → Step 10** (seat error → payment server error)

Performance vs. v33.0: Identical. No change in this app. Best-performing 411 case for both versions: 100% precision and recall with correctly structured chains.

Case 4 — Rent-a-Car App · 11 steps · Log: format=none (no unified log; inline logs for some steps)

Scenario: Verify main screen → select cities (Ankara→Izmir) → select dates (14–17 April) → search vehicles → verify vehicle list → click Kirala for Mercedes-Benz G-Class → navigate to payment screen → verify reservation details → enter promo code "DISCOUNT20" → click Uygula (apply) → click Onayla ve Öde (pay).

Ground truth per user annotation: Steps 5, 8, 9, and 10 are real failures. Step 7 (abstractVerify total ₺17,000) is a real pass. Step 8 fails because the discount was claimed applied by the UI but the total price did not update — a visual inconsistency the observer should have caught.

Step	Action / Element	System Result	Ground Truth	Assessment
0	abstractVerify — main screen loaded	PASS	Pass ✓	TN
1	click — Select cities Ankara/Izmir	PASS	Pass ✓	TN
2	click — Select dates 14/17 April	PASS	Pass ✓	TN
3	click — Search vehicles	PASS	Pass ✓	TN
4	abstractVerify — vehicle list displayed	PASS	Pass ✓	TN
5	click — Kirala (Mercedes-Benz G-Class)	FAIL (action_failed)	Fail ✗	TP ✓
6	navigate — to payment screen	PASS	Pass ✓	TN
7	abstractVerify — total ₺17,000 for 3 days	FAIL (content_mismatch)	Pass ✓	FP ✗
8	type — Promo code "DISCOUNT20"	PASS	Fail ✗	FN ✗
9	click — Uygula (apply discount)	FAIL (content_mismatch)	Fail ✗	TP ✓
10	click — Onayla ve Öde (pay)	FAIL (system_error, caused by Step 9)	Fail ✗	TP ✓

Result: 3 TP + 1 FP + 1 FN (vs. v33.0: 3 TP + 0 FP + 1 FN).

True Positive — Step 5: The Kirala action for Mercedes-Benz G-Class was blocked by a backend requirement: "special corporate account required for intercity drop-off in Izmir." Log observer flagged [SYS_ERR]. Classified as **action_failed** — visually the button appeared active but the action did not proceed.

True Positive — Step 9: The discount application showed the success message "**Tebrikler! %20 indirim uygulandı!**" but the total remained ₺17,000.00 unchanged. Classified as **content_mismatch**. Correctly linked as downstream of Step 5 (intercity restriction created inconsistent reservation state), and further linked to the FP at Step 7 in the formal chain.

True Positive — Step 10: POST /payment → 504 Gateway Timeout on Izmir regional gateway. Classified as **system_error**. Correctly chained from Step 9.

False Positive — Step 7 (New in v34.0): The abstractVerification step "verify that the 'Toplam Tutar' (total amount) is correct with the 3-day rental and insurance costs" was classified as **content_mismatch** with the reason: "The Observer did not confirm the total amount in POST, resulting in a content mismatch." This is incorrect — Step 7 is a **presence check** with no specific numeric expected value named in the instruction. The correct behavior (as in v33.0) is to set **expected_value** = 'N/A' and evaluate whether the Toplam Tutar element is visible (it is: ₺17,000.00 is displayed). The v34.0 system over-applies Rule 2 strict value verification, treating "Toplam Tutar'ın doğru olduğunu doğrula" as requiring numeric confirmation rather than a presence check. In G411 format, there is no separate **element** field — the full instruction text becomes the target, which contains verification language that misleads Rule 2. This FP was not present in v33.0.

False Negative — Step 8 (Unchanged from v33.0): Entering **DISCOUNT20** into the promo code field appeared visually successful — the text was correctly entered. Our system reported PASS. The ground truth requires FAIL because the discount was claimed applied by the UI (the success message "**Tebrikler! %20 indirim uygulandı!**" was visible) yet the total price of ₺17,000 remained unchanged in the same view. The Observer reported the promo code entry as successful without cross-checking whether the displayed total reflected the claimed discount. This is a **visual observation failure** identical to v33.0 — the evidence was on-screen but not caught.

Case 5 — Wikipedia Website · 5 steps · No log (format=none)

Scenario: Verify Vikipedi page present → click top-left menu → click Hakkımızda → click "138 dil" language option → click "Osmanlı padişahı 3. Murad tahta çıktı seç."

Ground truth per 411 reference output: Step 4 is a real failure (instruction target not present in any visible UI element).

Step	Action / Element	System Result	Ground Truth	Assessment
0	abstractVerify — "Vikipedi" visible	PASS	Pass ✓	TN
1	click — Top-left menu icon	PASS	Pass ✓	TN
2	click — Hakkımızda	PASS	Pass ✓	TN
3	click — "138 dil" option	PASS	Pass ✓	TN
4	click — Osmanlı padişahı 3. Murad...	FAIL (element_missing)	Fail X	TP ✓

Result: 1 failure detected, 1 real failure → Perfect precision and recall.

True Positive — Step 4: The instruction target "Osmanlı padişahı 3. Murad tahta çıktı seç." is lexically isolated from the current page context (Wikipedia Hakkımızda page). The element was absent from both PREV and POST screenshots. Classified as **element_missing**. The 411 reference classifies this as an "Instruction Error." Both systems agree: the target element is absent and the step fails. Operating in vision-only mode (no log data) was sufficient for this unambiguous absence detection.

Performance vs. v33.0: Identical. No change in this app.

Case 6 — Video App · 16 steps · Log: inline (g411 format)

Scenario: Verify app launch in Turkish → tap Profile tab → open Settings screen → change language to English → tap Back to Profile → tap Search tab → type 'Sci' in search bar → select 'Sci-Fi Collection' → tap Back → tap Home tab → tap 'Space Journey' series → open season selector → select Season 2 → tap Episode 1 → abstractVerify episode presence → abstractVerify episode description in English.

Ground truth: Step 15 is the only real failure — the language change did not propagate to the episode detail page.

Step	Action / Element	System Result	Ground Truth	Assessment
0	abstractVerify — app launched in Turkish	PASS	Pass ✓	TN
1	click — Profile tab	PASS	Pass ✓	TN
2	click — Ayarlar (Settings)	PASS	Pass ✓	TN
3	click — Language change to English	PASS	Pass ✓	TN
4	click — Back to Profile	PASS	Pass ✓	TN
5	click — Search tab	PASS	Pass ✓	TN
6	type — 'Sci' in search bar	PASS	Pass ✓	TN
7	click — Sci-Fi Collection	PASS	Pass ✓	TN
8	click — Back	PASS	Pass ✓	TN
9	click — Home tab	PASS	Pass ✓	TN
10	click — Space Journey series	PASS	Pass ✓	TN
11	click — Season selector	PASS	Pass ✓	TN
12	click — Season 2	PASS	Pass ✓	TN
13	click — Episode 1	PASS	Pass ✓	TN
14	abstractVerify — episode presence	PASS	Pass ✓	TN
15	abstractVerify — episode description in English	FAIL (content_mismatch)	Fail ✗	TP ✓

Result: 1 failure detected, 1 real failure → Perfect precision and recall.

True Positive — Step 15: The episode detail page displayed the description in Turkish despite the language change to English in Step 3. The system correctly classified this as `content_mismatch` — the expected behavior was English-language content, but the observed screen showed a Turkish description for the 'First Contact' episode. The root cause analysis identified a corrupted language setting: the language change did not propagate to the episode details page. Log evidence confirmed the language setting network request returned 200 OK, meaning the setting was saved but not globally applied at the client level. The causal analysis correctly flagged this as an isolated failure with no upstream cause in the current step chain.

Case 7 — Investment App · 15 steps · Log: inline (g411 format)

Scenario: Verify empty portfolio → navigate to Piyasalar (Markets) tab → search 'THY' in stocks → click THYAO stock → view 1-Month chart → click Buy button → enter quantity 100 → click Onayla (Confirm) [504 Timeout] → close success dialog → return to Piyasalar → navigate to Haberler (News) tab → open first news article → tap Back → navigate to Portföy (Portfolio) tab → abstractVerify THYAO shares in portfolio.

Ground truth: Steps 7 and 14 are real failures. The 504 Gateway Timeout at Step 7 silently prevented the trade from completing despite a success overlay; Step 14 confirms the portfolio is empty as a result.

Step	Action / Element	System Result	Ground Truth	Assessment
0	abstractVerify — portfolio empty	PASS	Pass ✓	TN
1	click — Piyasalar tab	PASS	Pass ✓	TN
2	type — 'THY' in search	PASS	Pass ✓	TN
3	click — THYAO stock	PASS	Pass ✓	TN
4	click — 1-Month chart button	PASS	Pass ✓	TN
5	click — Buy button	PASS	Pass ✓	TN
6	type — quantity 100	PASS	Pass ✓	TN
7	click — Onayla (Confirm order)	FAIL (system_error)	Fail ✗	TP ✓
8	click — Close success dialog	PASS	Pass ✓	TN
9	click — Piyasalar tab	PASS	Pass ✓	TN
10	click — Haberler tab	PASS	Pass ✓	TN
11	click — First news article	PASS	Pass ✓	TN
12	click — Back to news list	PASS	Pass ✓	TN
13	click — Portföy tab	PASS	Pass ✓	TN
14	abstractVerify — 100 THYAO shares in portfolio	FAIL (element_missing)	Fail ✗	TP ✓

Result: 2 failures detected, 2 real failures → Perfect precision and recall.

True Positive — Step 7 (Root Failure): The click on the Onayla (Confirm) button triggered a `POST /trade/order → 504 Gateway Timeout`. Log observer flagged `[NET_FAIL, SYS_ERR]`. The system correctly classified this as `system_error` despite the UI displaying a success overlay — the log evidence overrode the misleading visual result (log-override PASS→FAIL functioning as intended). This mirrors the O Bilet Step 7 and Bank App Step 4 patterns.

True Positive — Step 14 (Cascading Failure): The Portföy (Portfolio) screen returned an empty holdings array — no THYAO shares were found. Log confirmed a successful 200 OK response but with an empty holdings array, meaning the portfolio genuinely had no shares (the trade at Step 7 never completed). The

system correctly classified this as `element_missing` and the root cause chain analysis correctly linked [Step 7](#) → [Step 14](#) (network failure → empty portfolio), with confidence 0.95.

Causal chain correctly constructed: [Step 7](#) [`system_error`] → [Step 14](#) [`element_missing`] — the cleanest two-step chain in the Group 411 extended dataset.

Case 8 — Car App · 14 steps · Log: inline (g411 format) for select steps

Scenario: AbstractVerify app launched → click 5 Serisi model → click Configure → select M Sport Paket → abstractVerify package selected → click Continue → select 19-inch wheels → click Continue → select Taba Deri interior → click Continue → enable Direksiyon Isıtma toggle → click Özete Git (Go to Summary) → abstractVerify Sipariş Özeti screen → abstractVerify M Sport equipment in summary.

Ground truth: Only Step 13 is a real failure (M Sport not confirmed in summary). Steps 0 and 12 are false positives — both are abstract presence checks that the system misclassified as failures.

Step	Action / Element	System Result	Ground Truth	Assessment
0	abstractVerify — app launched	FAIL (element_missing)	Pass ✓	FP X
1	click — 5 Serisi model	PASS	Pass ✓	TN
2	click — Configure (Konfigüre Et)	PASS	Pass ✓	TN
3	click — M Sport Paket	PASS	Pass ✓	TN
4	abstractVerify — M Sport selected	PASS	Pass ✓	TN
5	click — Continue	PASS	Pass ✓	TN
6	click — 19-inch wheels	PASS	Pass ✓	TN
7	click — Continue (to interior)	PASS	Pass ✓	TN
8	click — Taba Deri interior	PASS	Pass ✓	TN
9	click — Continue (to extras)	PASS	Pass ✓	TN
10	click — Direksiyon Isıtma toggle	PASS	Pass ✓	TN
11	click — Özete Git (Go to Summary)	PASS	Pass ✓	TN
12	abstractVerify — Sipariş Özeti screen	FAIL (content_mismatch)	Pass ✓	FP X
13	abstractVerify — M Sport in summary	FAIL (content_mismatch)	Fail X	TP ✓

Result: 1 TP + 2 FP + 0 FN.

False Positive — Step 0: The abstractVerification step "Uygulamanın açıldığını doğrula" (Verify the app is opened) was classified as `element_missing`. The logs explicitly confirmed successful app launch, and the POST screen clearly shows the car model list — the app is open. The system attempted to locate the

instruction text "Uygulamanın açıldığını doğrula" as a literal UI element on screen rather than treating it as an abstract state assertion. The correct outcome is Rule 3 SUCCESS: the presence check is satisfied by the visible app interface. This is an instance of the **abstract state verification FP pattern** — the instruction describes a system state, not a named UI element.

False Positive — Step 12: The abstractVerification of the Sipariş Özeti (Order Summary) screen was classified as `content_mismatch` with "VALUE NOT CONFIRMED IN POST." The screen visibly shows the Sipariş Özeti screen (correctly reached after Step 11's click on Özete Git). This is a screen presence check, not a data value assertion — `expected_value` should be 'N/A'. The system incorrectly applied Rule 2 strict value verification to a navigation-outcome check. This is the same Rule 2 over-application pattern as Rent-a-Car Case 4 Step 7 (FP 2).

True Positive — Step 13: The abstractVerification of M Sport equipment in the summary screen correctly detected a failure: the Sipariş Özeti screen showed standard equipment and 19-inch wheels but did not confirm the 'M Sport Paket' selection made at Step 3. Classified as `content_mismatch`. This is a genuine application bug — the configuration was made but the summary did not reflect it.

Sub-optimality — FP-rooted causal chain: The system built the chain `Step 0 (FP) → Step 12 (FP) → Step 13 (TP)`. Since Steps 0 and 12 are both FPs, the entire chain root and intermediate node are incorrect. Step 13's true failure (M Sport absent from summary) is likely an independent application bug unrelated to the FP-rooted chain. The formal chain builder incorrectly attributed Step 13's failure to the preceding FPs.

Case 9 — Hotel Booking App · 16 steps · Log: inline (g411 format)

Scenario: App launch → type 'Antalya' in destination field → select destination → select dates Aug 10–15 → open guest picker → confirm 2 adults 2 children [backend saves 1 adult 0 children] → click Otel Bul (Hotel Search) → apply 5-star filter → abstractVerify hotels loaded → click first hotel → view photo gallery → close gallery → navigate to room selection → select Aile Süiti (Family Suite) → click Rezervasyona Geç (Proceed to Reservation) → abstractVerify guest count in summary.

Ground truth: Steps 5 and 15 are real failures (guest selection backend mismatch and downstream summary error). Step 8 is a false positive.

Step	Action / Element	System Result	Ground Truth	Assessment
0	abstractVerify — app launched	PASS	Pass ✓	TN
1	type — 'Antalya' in destination	PASS	Pass ✓	TN
2	click — Select destination	PASS	Pass ✓	TN
3	click — Select dates Aug 10–15	PASS	Pass ✓	TN
4	click — Open guest picker	PASS	Pass ✓	TN
5	click — Confirm '2 Yetişkin, 2 Çocuk'	FAIL (content_mismatch)	Fail ✗	TP ✓
6	click — Otel Bul (Hotel Search)	PASS	Pass ✓	TN

Step	Action / Element	System Result	Ground Truth	Assessment
7	click — 5-star filter	PASS	Pass ✓	TN
8	abstractVerify — hotels loaded	FAIL (element_missing)	Pass ✓	FP X
9	click — First hotel in list	PASS	Pass ✓	TN
10	click — Fotoğrafları Gör (View Photos)	PASS	Pass ✓	TN
11	click — Close gallery	PASS	Pass ✓	TN
12	click — Room selection	PASS	Pass ✓	TN
13	click — Aile Süiti (Family Suite)	PASS	Pass ✓	TN
14	click — Rezervasyona Geç	PASS	Pass ✓	TN
15	abstractVerify — guest count '2 Yetişkin, 2 Çocuk'	FAIL (content_mismatch)	Fail X	TP ✓

Result: 2 TP + 1 FP + 0 FN.

True Positive — Step 5 (Root Failure): The guest confirmation UI displayed '2 Yetişkin, 2 Çocuk' (2 Adults, 2 Children) as selected, but the backend POST payload saved only 1 adult and 0 children — a direct UI-backend data mismatch. Log observer confirmed: the POST returned 200 OK but with an incorrect payload. The system correctly classified this as **content_mismatch**. This type of failure (UI shows correct state, backend stores incorrect data) is particularly dangerous because visual-only checks would miss it; log evidence was decisive.

True Positive — Step 15 (Downstream Failure): The reservation summary screen displayed '1 Yetişkin' (1 Adult) instead of the expected '2 Yetişkin, 2 Çocuk'. The system correctly classified this as **content_mismatch** and identified it as downstream of Step 5's data mismatch — the corrupted guest count saved at Step 5 propagated to the summary display. The root cause chain correctly linked **Step 5 → Step 15**.

False Positive — Step 8: The abstractVerification step "Otellerin yüklendiğini doğrula" (Verify hotels are loaded) was classified as **element_missing**. The hotels ARE visible and functional — Step 9 successfully clicks the first hotel in the list, proving hotels were displayed. The system attempted to match "Otellerin yüklendiğini doğrula" as a literal UI element name, rather than treating it as an abstract state check ("are hotels shown?"). The correct outcome is Rule 3 SUCCESS. This is the same **abstract state verification FP pattern** as Case 8 Step 0.

Sub-optimality — FP-rooted causal chain: The system's root cause analysis constructed the chain **Step 5 (TP) → Step 8 (FP) → Step 15 (TP)**. The FP at Step 8 distorts the causal path — the true chain is **Step 5 → Step 15** directly (backend saved wrong guest count → summary shows wrong count). Step 8 is not a real intermediate failure.

Case 10 — Clothing App · 12 steps · Log: inline (g411 format)

Scenario: App launch → type 'Erkek Tişört' in search → click Search button → click Filtrele (Filter) → select 'Siyah' (Black) color filter → select 'M' size filter → select 'Nike' brand → click Sonuçları Göster (Show Results) →

abstractVerify filtered results → click first product → abstractVerify product details → abstractVerify Siyah/M attributes on product page.

Ground truth: Step 11 is the only real failure — the product detail page showed a different product (Kırmızı/XL) than the filters applied (Siyah/M).

Step	Action / Element	System Result	Ground Truth	Assessment
0	abstractVerify — app launched	PASS	Pass ✓	TN
1	type — 'Erkek Tişört' in search bar	PASS	Pass ✓	TN
2	click — Search button	PASS	Pass ✓	TN
3	click — Filtrele (Filter)	PASS	Pass ✓	TN
4	click — Siyah color filter	PASS	Pass ✓	TN
5	click — M size filter	PASS	Pass ✓	TN
6	click — Nike brand filter	PASS	Pass ✓	TN
7	click — Sonuçları Göster (Show Results)	PASS	Pass ✓	TN
8	abstractVerify — filtered results displayed	PASS	Pass ✓	TN
9	click — First product in list	PASS	Pass ✓	TN
10	abstractVerify — product detail page	PASS	Pass ✓	TN
11	abstractVerify — color 'Siyah', size 'M'	FAIL (content_mismatch)	Fail X	TP ✓

Result: 1 failure detected, 1 real failure → Perfect precision and recall.

True Positive — Step 11: The product detail page displayed color 'Kırmızı' (Red) and size 'XL' instead of the expected 'Siyah' (Black) and 'M' from the applied filters. The system correctly classified this as `content_mismatch`. Log confirmed the product was rendered with product id 998877 (color: red, size: XL) — inconsistent with the Siyah/M/Nike filters applied in Steps 4–6. The root cause analysis identified a corrupted product detail display state: the filters were applied to the search results but the clicked product did not reflect the filtered attributes on its detail page. Classified as an isolated failure with no upstream cause in the current step chain.

5. Precision, Recall & F1 by App

5A. Our Own Traces

App	Analyzed Steps	GT Failures	Predicted	TP	FP	FN	TN	Precision	Recall	F1
Migros	8	1	1	1	0	0	7	100%	100%	100%
Bitaksi	8	0	0	0	0	0	8	N/A	N/A	N/A
Clock	7	0	0	0	0	0	7	N/A	N/A	N/A
Flight App	22	6	7	6	1	0	15	85.7%	100%	92.3%
QNB	13	1	1	1	0	0	12	100%	100%	100%
Yemeksepeti	14	2	2	2	0	0	12	100%	100%	100%
O Bilet	9	2	2	2	0	0	7	100%	100%	100%
TOTAL	81	12	13	12	1	0	68	92.3%	100.0%	96.0%

For apps with 0 ground truth failures, precision is undefined (no positive examples). Bitaksi and Clock contribute 0 TP and 0 FP — pure TN performance.

5B. Group 411 Traces

App	Analyzed Steps	GT Failures	Predicted	TP	FP	FN	TN	Precision	Recall	F1
Passo	11	1	1	1	0	0	10	100.0%	100.0%	100.0%
Bank App	8	3	2	2	0	1	5	100.0%	66.7%	80.0%
Ticket App	11	4	4	4	0	0	7	100.0%	100.0%	100.0%
Rent-a-Car	11	4	4	3	1	1	6	75.0%	75.0%	75.0%
Wikipedia	5	1	1	1	0	0	4	100.0%	100.0%	100.0%
Video App	16	1	1	1	0	0	15	100.0%	100.0%	100.0%
Investment App	15	2	2	2	0	0	13	100.0%	100.0%	100.0%
Car App	14	1	3	1	2	0	11	33.3%	100.0%	50.0%
Hotel Booking App	16	2	3	2	1	0	13	66.7%	100.0%	80.0%
Clothing App	12	1	1	1	0	0	11	100.0%	100.0%	100.0%
TOTAL	119	20	22	18	4	2	95	81.8%	90.0%	85.7%

5C. Combined Summary

Dataset	Analyzed	GT Fail	Predicted	TP	FP	FN	TN	Precision	Recall	F1
Our Traces	81	12	13	12	1	0	68	92.3%	100.0%	96.0%
Group 411	119	20	22	18	4	2	95	81.8%	90.0%	85.7%
COMBINED	200	32	35	30	5	2	163	85.7%	93.8%	89.6%

6. Error Classification

6.1 False Positives (5 total)

FP #	App	Step	Trigger	Root Cause Category
1	Flight App	Step 19	Vision observer reported click on BASIC package option (within wrong-flight class overlay) produced no state change or navigation; GT = Pass	GPT-4o non-determinism — Decider applied Rule 1 (no visible state change) instead of Rule 3 CONTENT EXPANSION SUCCESS; the flight class overlay in POST is new content signaling the click was registered, but the Decider failed to recognize the overlay interaction as a valid action response
2	Rent-a-Car	Step 7	AbstractVerification "Toplam Tutar'ın doğru olduğunu doğrula" classified as content_mismatch because Observer did not explicitly confirm a numeric value; GT = Pass	Rule 2 over-application — no specific numeric expected value named in instruction; "verify total is correct" is a presence check (expected_value = N/A), not strict value verification; G411 format has no separate element field, causing the full instruction text (with verification language) to trigger Rule 2 incorrectly
3	Car App	Step 0	AbstractVerification "Uygulamanın açıldığını doğrula" classified as element_missing; GT = Pass	Abstract state verification misclassification — the instruction describes an app-level state ("app is open"), not a named UI element; the system attempted to locate the instruction text as a literal element on screen; the visible car model list and log confirmation of app launch are sufficient for Rule 3 SUCCESS
4	Car App	Step 12	AbstractVerification of Sipariş Özeti screen classified as content_mismatch ("VALUE NOT CONFIRMED IN POST"); GT = Pass	Rule 2 over-application — the Sipariş Özeti screen IS visible (navigation to it succeeded at Step 11); this is a screen presence check with no specific data value assertion; same pattern as FP 2 (Rent-a-Car Step 7)

FP #	App	Step	Trigger	Root Cause Category
5	Hotel Booking App	Step 8	AbstractVerification "Otellerin yüklendiğini doğrula" classified as element_missing; GT = Pass	Abstract state verification misclassification — "hotels are loaded" is a system state assertion, not a named UI element; hotels ARE visible (Step 9 successfully clicks the first hotel); same pattern as FP 3 (Car App Step 0)

Analysis — FP 1 (Flight Step 19): The false positive originates in the Logic Decider, not the Vision Observer. The BASIC package option was part of an overlay that appeared in POST after Step 18's click — the overlay itself IS new content (Rule 3 CONTENT EXPANSION SUCCESS should apply to the Step 18 click, and Step 19's click within that overlay constitutes a registered action). The Decider instead applied Rule 1 constraints to Step 19 as if the overlay was the expected final state rather than an intermediate state. This regression is non-deterministic: the identical scenario correctly passes in v33.0. The root cause is that the Rule 3 CONTENT EXPANSION SUCCESS clause does not explicitly cover the case where a step's target is within a content-expansion overlay produced by a prior step — making the boundary between expansion-content recognition and state-change detection ambiguous for GPT-4o at temperature=0.

Analysis — FP 2 (Rent-a-Car Step 7) and FP 4 (Car App Step 12): Both originate in Rule 2 application logic. Instructions containing "doğrula" (verify/confirm) alongside an element name are interpreted as requiring explicit numeric confirmation, even when no specific numeric expected value is named. The correct handling is to treat these as presence checks with `expected_value = 'N/A'`. The regression may be linked to prompt changes in the Rule 2 STEP TYPE GATE that affected how the presence vs. value-assertion distinction is applied to G411-format instructions lacking a structured `element` field.

Analysis — FP 3 (Car App Step 0) and FP 5 (Hotel Booking Step 8): Both originate from a new pattern in the extended dataset: **abstract state/presence verification instructions** where the instruction text describes a system-level state (e.g., "app is opened", "hotels are loaded") rather than a named UI element. The Decider applies Rule 1 (element_missing) because it cannot match the instruction text to any visible UI element. The correct behavior is to evaluate whether the described state is evident from the visual context (app interface visible → app is open; hotel list visible in next step → hotels were loaded). A structural fix requires distinguishing "state description" instructions from "element presence" instructions at the Rule 1 gate.

6.2 False Negatives (2 total — unchanged from v33.0)

FN #	App	Step	System Result	Ground Truth	Root Cause Category
1	Bank App	Step 6	PASS (dashboard loaded)	FAIL (balance unchanged after "successful" transfer)	Cross-step state inconsistency — requires tracking expected state across non-adjacent steps (Step 0 balance, Step 3 amount, Step 5 success claim, Step 6 balance comparison)
2	Rent-a-Car	Step 8	PASS (promo code entered)	FAIL (discount claimed applied)	Visual observation failure — discount success message and unchanged ₺17,000 total were both visible in POST; observer

FN #	App	Step	System Result	Ground Truth	Root Cause Category
				but total price unchanged)	reported text entry success without verifying the price reflected the claimed discount

FN Pattern Analysis (Unchanged from v33.0):

- **Bank App Step 6** — failure produces no single-step visual evidence; only detectable by comparing the balance at Step 6 against the expected post-transfer value derived from Steps 0 and 3. Genuine **cross-step state inconsistency** requiring expected-state propagation.
- **Rent-a-Car Step 8** — the failure was visually detectable within the step itself: the UI simultaneously displayed a discount-applied success message and an unchanged total price of ₺17,000. This is a **visual observation failure** — the Vision Observer focused on confirming promo code entry rather than verifying price-discount consistency.

No new FNs in Cases 6–10. All 7 real failures across the new cases were correctly detected.

6.3 Sub-optimality (Not Detection Errors)

Issue	App	Description
Missed causal chain	Yemeksepeti	Steps 7 and 14 both detected but reported as ISOLATED rather than a chain (Step 7 → Step 14). <code>find_likely_cause()</code> scored below 0.3 due to 6-step temporal gap and click vs. abstractVerification type mismatch. Root cause analysis section correctly identifies the link but the formal chain builder does not. Both steps were correctly detected. Unchanged from v33.0.
FP-rooted causal chain	Rent-a-Car	The formal chain Step 5 → Step 7 (FP) → Step 9 → Step 10 is constructed. The inclusion of the FP at Step 7 means the chain path traverses an incorrect node. The actual causal root (Step 5: corporate account blockage) is correctly identified, but the chain is distorted by the Step 7 FP. The true chain should be Step 5 → Step 9 → Step 10 (as in v33.0).
Bank FN cascade	Bank App	Step 7 (content_mismatch) is downstream of Step 6 (FN). Because Step 6 was missed, the chain Step 6 → Step 7 is not constructed. The root cause analysis does link Step 4 → Step 7, which partially compensates. Unchanged from v33.0.
FP-rooted causal chain	Car App (Case 8)	The system built chain Step 0 (FP) → Step 12 (FP) → Step 13 (TP) . Both the root and intermediate nodes are FPs. Step 13's real failure (M Sport absent from summary) is likely independent of the FP-rooted chain. The chain builder incorrectly attributed the TP failure to the preceding abstract-check FPs.
FP-rooted causal chain	Hotel Booking App (Case 9)	The root cause chain Step 5 (TP) → Step 8 (FP) → Step 15 (TP) includes the FP at Step 8 as an intermediate node. The true chain is Step 5 → Step 15 directly (backend saved wrong guest count → summary shows

Issue	App	Description
		wrong count). The FP distorts the chain path but both real failures (Steps 5 and 15) are correctly detected.

7. Key Takeaways

7.1 Changes from v33.0 to v34.0

Issue	v33.0 Count	v34.0 Count	Status
Passo FP (Step 5) — content expansion VLM observation error	1 FP	0	Fixed ✓
Flight App FP (Step 19) — BASIC overlay click non-deterministic misclassification	0 FP	1	New regression
Rent-a-Car FP (Step 7) — Rule 2 over-application to presence-check instruction	0 FP	1	New regression
Car App FP (Step 0) — abstract state instruction misclassified as element_missing	0 FP	1	New FP (Cases 6–10)
Car App FP (Step 12) — Rule 2 over-application to screen presence check	0 FP	1	New FP (Cases 6–10)
Hotel Booking FP (Step 8) — abstract state instruction misclassified as element_missing	0 FP	1	New FP (Cases 6–10)
Missed causal chain (Yemeksepeti 7→14)	1	1	Unchanged (sub-optimality)
Cross-step state FN (Bank App Step 6)	1 FN	1	Unchanged (architectural gap)
Price-discount consistency FN (Rent-a-Car Step 8)	1 FN	1	Unchanged (observer prompt gap)
Total FPs	1	5	+4
Total FNs	2	2	No change

7.2 Performance Dimensions Summary

Dimension	Our Traces	Group 411	Combined	Verdict
Recall / failure detection	100%	90.0%	93.8%	Perfect on owned data; strong on external
Precision / noise	92.3%	81.8%	85.7%	Slight regression on owned; new FP pattern on external

Dimension	Our Traces	Group 411	Combined	Verdict
F1 Score	96.0%	85.7%	89.6%	High across both datasets
Log-integrated detection	100% (with logs)	91.7% (original 5)	—	Log evidence consistently improves recall
Vision-only detection	100% (Yemeksepeti)	100% (Wikipedia, Video App)	—	VLM sufficient when visual signal is strong
Causal chain accuracy	1 FP-rooted chain	3 FP-rooted chains (RC, Car App, Hotel App)	—	Regression: FP-rooted chains distort paths but TPs detected
Cross-step consistency	Not applicable	0% (2 FNs)	—	Current architectural gap — unchanged
Abstract state check handling	N/A	0% (3 FPs in Cases 8–9)	—	New gap: abstract state instructions misclassified

7.3 Generalization Assessment

The system was developed and tuned on the proprietary traces. Applied to all 10 Group 411 external test cases, v34.0 maintains strong generalization: **90.0% recall** (all real failures in 8 of 10 cases detected; 0 new FNs in Cases 6–10) with **81.8% precision** across the full 10-case Group 411 dataset.

The 3 new FPs in Cases 8 and 9 share a common pattern — **abstract state/presence verification instructions** — that was not prominent in the original 5 Group 411 cases. This pattern emerges when G411-format test cases use instructions that describe system states ("app is opened", "hotels are loaded") rather than named UI elements, and the Decider applies Rule 1 (element_missing) instead of a presence-check Rule 3 pass. This is a systematic prompt gap, not GPT-4o non-determinism.

Cases 6, 7, and 10 achieve 100%/100%/100% — demonstrating that when instructions reference named UI elements or data values, the system generalizes cleanly to new domains (streaming apps, investment apps, fashion apps). The new FP pattern is bounded: it only triggers on abstract state assertion instructions in G411 format.

The 2 original FNs (Bank App Step 6, Rent-a-Car Step 8) remain structurally unchanged — both require capabilities not yet implemented (cross-step state tracking; price-discount trigger-reveal cross-checking). These are genuine architectural limitations, not prompt failures.

8. Recommendations

Rec 1 — Multi-step state propagation for abstractVerification steps (addresses Bank App FN — unchanged from v33.0)

Both false negatives ultimately require comparing a value from a previous step against the current step's observations. Implement a **state scratchpad** in **GraphState** that records key numerical values extracted by the Vision Observer from prior steps (balances, amounts, totals). When the Logic Decider encounters a step where the **action** is **abstractVerification** and the instruction references a prior value, inject the relevant

scratchpad entries as [STATE CONTEXT]. This would enable cross-step inconsistency detection without restructuring the pipeline.

Estimated impact: Eliminates FN at Bank App Step 6. Also lays groundwork for Rent-a-Car Step 8 if the expected post-discount total is tracked from prior steps.

Rec 2 — Rule 2 gate: distinguish presence checks from value assertions (addresses Rent-a-Car Step 7 FP and Car App Step 12 FP)

The Rule 2 STEP TYPE GATE must reliably distinguish "verify X equals Y" (specific data value assertion) from "verify X is correct/present" (presence or reasonableness check). For G411-format steps where the instruction carries verification language without a quoted or structured expected value, set `expected_value = 'N/A'` before entering Rule 2. A concrete heuristic: if the instruction contains "verify/confirm/check [element] is correct/present/visible" without a quoted numeric or formatted string, treat as presence check and skip to Rule 3. Prompt-based gates have shown to cause regressions (e.g., the reverted CRITICAL NO-VALUE GATE in this development cycle); a structural solution that parses the instruction's value-reference pattern before applying Rule 2 is recommended.

Estimated impact: Eliminates FP 2 (Rent-a-Car Step 7) and FP 4 (Car App Step 12). Reduces risk of similar FPs on G411-format steps with "doğrula" / "verify is correct" phrasing.

Rec 3 — Rule 1 gate: distinguish named UI element targets from abstract state assertions (addresses Car App Step 0 FP and Hotel Booking Step 8 FP)

Add a pre-check to Rule 1 (TARGET ABSENT) that classifies the instruction target as either a **named UI element** (e.g., a button label, screen title, specific text) or an **abstract state description** (e.g., "app is opened", "hotels are loaded", "screen is displayed"). For abstract state descriptions, skip Rule 1 and evaluate using visible context evidence: if the POST screen plausibly supports the described state (app interface visible → app is open; hotel list visible → hotels loaded), apply Rule 3 SUCCESS. This directly targets FPs 3 and 5 where instructions describe system states that are trivially satisfied by the visible screen context.

Estimated impact: Eliminates FP 3 (Car App Step 0) and FP 5 (Hotel Booking Step 8). Reduces sensitivity to the abstract state instruction pattern that appeared in 2 of the 5 new Group 411 cases.

Rec 4 — Rule 3 CONTENT EXPANSION SUCCESS: explicit overlay-interaction clause (addresses Flight Step 19 FP)

Add an explicit note to Rule 3 CONTENT EXPANSION SUCCESS covering the case where a step's target is within a content-expansion overlay produced by a prior step: "If POST shows the same overlay or content panel that appeared in the prior step's POST (as new content), and the instruction targets an element within that panel, a click that does not cause further navigation but produces no negative state change is a SUCCESS — the overlay received the interaction. Do NOT classify as `action_failed`." This directly targets Step 19 (BASIC in flight class overlay) and prevents similar FPs on any multi-step overlay interaction.

Estimated impact: Eliminates the non-deterministic Flight App Step 19 FP. Reduces sensitivity to GPT-4o temperature=0 variance in overlay-click step classification.

Rec 5 — Observer prompt: verify price-discount consistency for promo code steps (addresses Rent-a-Car Step 8 FN — unchanged from v33.0)

When a step involves entering a promo code, coupon, or discount value, the Vision Observer should explicitly check whether the displayed total price is consistent with the claimed discount. Add a clause: "If the POST screen shows a discount success message alongside a price or total value, verify that the displayed total reflects the stated discount percentage applied to the pre-discount amount. If the total is unchanged despite the success message, report this as a price-discount inconsistency." This targets the Step 8 miss — both the discount message and the unchanged ₺17,000 were on-screen but the observer did not cross-check them.

Rec 6 — Strengthen causal chain scoring for abstractVerification steps (addresses Yemeksepeti sub-optimality — unchanged from v33.0)

abstractVerification steps that fail should be treated as likely downstream of any earlier **content_mismatch** failure involving the same logical entity. Add a keyword-similarity check between the failed **abstractVerification** instruction and prior failure **root_cause** strings. A cosine similarity above 0.5 should override the 0.3 threshold in **find_likely_cause()** when the types are **click** → **abstractVerification**. This was identified in v33.0 and remains relevant.

Rec 7 — Confidence distribution calibration (unchanged from v33.0)

All analyzed steps in this evaluation were classified as High confidence (≥ 0.85). Given that 7 steps were incorrectly evaluated (5 FP, 2 FN), confidence does not distinguish correct from incorrect predictions. Consider adding a calibration pass that penalizes confidence when: (a) log data is absent (**has_logs=False**) for action types that typically generate backend events, or (b) the visual observer's **semantic_screen_summary** contradicts its **post_screen_changes** at element level. Calibrated confidence would allow downstream consumers to identify uncertain steps rather than treating all high-confidence outputs uniformly.

9. Dataset Coverage Summary

App / Case	Source	Log Format	Mode	Steps	GT Fail	TP	FP	FN
Migros	Our traces	nested	preassigned	8	1	1	0	0
Bitaksi	Our traces	flat_assertions	preassigned	8 (1 skip)	0	0	0	0
Clock	Our traces	flat_indexed	preassigned	7	0	0	0	0
Flight App	Our traces	raw	dynamic/keyword	22 (3 skip)	6	6	1	0
QNB	Our traces	flat_indexed	preassigned	13	1	1	0	0
Yemeksepeti	Our traces	nested (empty)	preassigned	14 (1 skip)	2	2	0	0
O Bilet	Our traces	nested	preassigned	9	2	2	0	0

App / Case	Source	Log Format	Mode	Steps	GT Fail	TP	FP	FN
Passo App	Group 411	inline g411 / none	per-step	11	1	1	0	0
Bank App	Group 411	inline g411 / none	per-step	8	3	2	0	1
Ticket App	Group 411	inline g411	per-step	11	4	4	0	0
Rent-a-Car	Group 411	inline g411 / none	per-step	11	4	3	1	1
Wikipedia	Group 411	none	none	5	1	1	0	0
Video App	Group 411	inline g411	per-step	16	1	1	0	0
Investment App	Group 411	inline g411	per-step	15	2	2	0	0
Car App	Group 411	inline g411 / none	per-step	14	1	1	2	0
Hotel Booking App	Group 411	inline g411	per-step	16	2	2	1	0
Clothing App	Group 411	inline g411	per-step	12	1	1	0	0
TOTAL				200	32	30	5	2

Report generated for Describer-Decider v34.0. Evaluation date: 2026-05-24. Our traces: 7 proprietary apps across Turkish e-commerce, transport, finance, and food delivery domains. Group 411 traces: 10 independently designed test cases (Passo ticketing, bank transfer, event ticketing, rent-a-car, Wikipedia web, video streaming, investment/stock trading, car configurator, hotel booking, clothing/fashion) Output source: *last fix current output.txt* (commit c8c2efe — v33.0 + BUTTON DISMISSED fix + Fix 1 + Fix 2) and *case6-10 outputs.txt* (Cases 6–10 extended evaluation)